RESUME ANALYZER AI AGENT:

AN AI-ASSISTED SYSTEM FOR RESUME–JOB MATCHING, SKILL-GAP ANALYSIS,LEARNING-PATH PLANNING, AND INTERVIEW-ROUTE OPTIMIZATION

A MICRO PROJECT REPORT

Submitted by

SWATHI M	**2303717610422125**
SWETHA M	**2303717610422126**

in partial fulfillment of the requirements of the laboratory course

ARTIFICIAL INTELLIGENCE LABORATORY

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

COIMBATORE-641014

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

ABSTRACT

The Resume Analyzer AI Project is an integrated application designed to help candidates evaluate resume compatibility with job descriptions, identify skill gaps, plan personalized learning paths, estimate study duration, and organize travel between interview locations. The system accepts resumes in PDF, DOCX, and plain-text formats, extracts their textual content, and analyzes them against a target job description. By combining text-vector similarity with Large Language Model (LLM)-based structured information extraction, the application generates resume match scores, identifies matching and missing skills, and provides suggestions for resume improvement.

A structured skill dataset stores learning effort estimates, skill categories, and prerequisite relationships. These relationships are used to analyze skill gaps and generate learning paths using Hill Climbing and A* search algorithms. The system also supports interview travel planning by calculating geographical distances between locations using the Haversine formula and determining suitable visit orders using the Nearest Neighbor algorithm, brute-force permutation search, or Held–Karp dynamic programming, depending on the number of interview locations.

The application presents its results through a Streamlit-based user interface, including resume rankings, skill-gap reports, learning timelines, and interactive route visualizations. By integrating resume analysis, intelligent learning-path planning and interview travel optimization, the project provides a comprehensive decision-support system for career preparation. The accuracy of its results depends on the quality of the input.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF TABLES	iv
	LIST OF FIGURES	iv
	LIST OF ABBREVIATIONS	v
1	**INTRODUCTION**	1
	1.1 BACKGROUND OF THE PROJECT DOMAIN	1
	1.2 PROBLEM STATEMENT	1
	1.3 OBJECTIVES OF THE PROJECT	2
	1.4 APPLICATIONS	2
	1.5 SCOPE OF THE PROJECT	3
2	**SYSTEM ARCHITECTURE**	4
	2.1 PROPOSED SYSTEM ARCHITECTURE	4
3	**SYSTEM REQUIREMENTS**	8
	3.1 HARDWARE REQUIREMENTS	8
	3.2 SOFTWARE REQUIREMENTS	8
4	**SYSTEM DESIGN AND IMPLEMENTATION**	9
	4.1 SYSTEM MODULES	9
	4.2 TECHNOLOGY STACK	11
	4.3 ALGORITHM AND TECHNIQUE USED	14
	4.4 DATA AND INDEX DESIGN	14
	4.5 RELIABILITY, PRIVACY AND LIMITATION	15
	4.6 TESTING AND EVALUATION	15
5	**RESULTS AND DISCUSSIONS**	16
	5.1 RESUME UPLOAD	16
	5.2 DATA LOADER	16
	5.3 JD MATCHER	17
	5.4 MATCHING SCORE	18
	5.5 SUGGESTOR	18
	5.6 CHATBOT	19
	5.7 GAPS FINDER AND OPTIMAL STUDY PLAN	20
	5.8 PATH EXPLORED	20

CHAPTER NO.	TITLE	PAGE NO.
	5.9 INTERVIEW OPTIMED ROUTE FINDER	21
	5.10 MAP	21
6	**CONCLUSION**	22
	6.1 CONCLUSION	22

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
3.1	HARDWARE REQUIREMENTS	8
3.2	SOFTWARE REQUIREMENTS	8
4.2	TECH STACK	13
4.3	ALGORITHS AND TECHNIQUES USED	**Error! Bookmark not defined.**
4.4	DATA AND INDEX DESIGN	14
4.5	RELIABILITY,PRIVACY AND LIMITATIONS	15
4.6	TESTING AND EVALUATION	15

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
2.1	PROPOSED SYSTEM ARCHITECTURE	4
5.1	RESUME UPLOAD	16
5.2	DATA LOADED	**Error! Bookmark not defined.**
5.3	JD MATCHER	**Error! Bookmark not defined.**
5.4	MATCHING SCORE	**Error! Bookmark not defined.**

FIGURE NO	TITLE	PAGE NO
5.5	SUGGESTOR	**Error! Bookmark not defined.**
5.6	CHATBOT	**Error! Bookmark not defined.**
5.7	GAPS FINDER AND OPTIMAL STUDY PLAN	**Error! Bookmark not defined.**
5.8	PATH EXPLORED	20
5.9	INTERVIEW OPTIMIZED ROUTE FINDER	21
5.10	MAP	21

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
A*	A-star search algorithm
API	Application Programming Interface
DOCX	Microsoft Word Open XML document format
DP	Dynamic Programming
JD	Job Description
LLM	Large Language Model
PDF	Portable Document Format
TF-IDF	Term Frequency–Inverse Document Frequency
TSP	Traveling Salesman Problem
TXT	Plain-text file format
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND OF THE PROJECT DOMAIN

Recruitment and job preparation require candidates to understand employer requirements, communicate relevant experience, identify missing competencies, and plan how to acquire them. Comparing a resume with a job description manually can be time-consuming, especially when multiple candidates or multiple requirements must be considered. Candidates may also struggle to prioritize skills and estimate the time needed to prepare for a role. For interviews conducted at different locations, travel planning introduces an additional optimization problem.

The Resume Analyzer AI Project brings these tasks together in one workflow. It processes candidate resumes and a target job description, calculates similarity and extracts structured matching information, determines skill gaps, generates a proposed learning sequence, and optimizes the order of interview visits. The project combines document parsing, natural-language processing, graph-based dependency modelling, search algorithms, and geographic distance calculations.

The system supports analysis and planning rather than making a final hiring decision. A numerical match score should be interpreted as an estimate of alignment with the supplied job description, not as a definitive measure of a candidate's ability or suitability.

1.2 PROBLEM STATEMENT

Candidates often lack a single, consistent tool that can compare their resumes against a job description, rank multiple resumes, explain matching and missing skills, estimate the effort required to learn missing

skills, and plan travel across interview venues. Manual evaluation is repetitive and can overlook relevant terms or prerequisite relationships. Informal learning plans may also ignore dependencies between skills, while manually ordering several interview locations may lead to unnecessarily long routes.

The problem addressed by this project is to develop an AI-assisted system that accepts resume and job information, generates explainable matching outputs, identifies skill gaps and a feasible learning order, estimates study effort using a configured skills dataset, and proposes an efficient route through multiple interview locations.

1.3 OBJECTIVES OF THE PROJECT

- Parse resumes supplied as PDF, DOCX, or plain text into usable text.
- Compare one or more resumes against a target job description and rank candidates by estimated match.
- Extract matching skills, missing skills, and resume-improvement recommendations using an LLM-based workflow.
- Identify missing skills and expand their prerequisite dependencies using a structured skill dataset.
- Generate a learning order using a greedy Hill Climbing approach and an A* search approach.
- Estimate total learning effort from the configured effort-days values.
- Calculate great-circle distances between interview locations from latitude and longitude.
- Find a short interview route using Nearest Neighbor, brute force, or Held–Karp dynamic programming.

- Present results in a Streamlit user interface.

1.4 APPLICATIONS

- Recruitment support: shortlist resumes that appear relevant to a job description for human review.
- Career guidance: identify skills a candidate may need to develop for a target role.
- Resume improvement: locate missing or weakly represented skills and experience.
- Campus placement preparation: compare student resumes with entry-level job descriptions.
- Resume search: retrieve relevant resumes from a collection using natural-language queries.

1.5 SCOPE OF THE PROJECT

The described system is intended for PDF resumes with selectable/extractable text, a user-supplied job description, and an internet connection for hosted HuggingFace LLM inference.

The design supports comparison and information retrieval, not automatic hiring decisions. It should not infer protected characteristics or use them to rank candidates. Results depend on the completeness of the resume, PDF extraction quality, chunking strategy, prompts, embedding model, and LLM response quality.

CHAPTER 2
SYSTEM ARCHITECTURE

2.1 PROPOSED SYSTEM ARCHITECTURE

Figure 2.1 illustrates an end-to-end automated talent pipeline and learning optimization system divided into five coordinated architectural phases. The process begins with Input Data ingestion and Natural Language Processing & Embeddings, where raw candidate resumes and locations are processed via LLMs to extract skills and compute matching metrics. Next, it performs a Skill Gap Analysis by cross-referencing candidate data against a structured Skill Knowledge Graph using set difference operations to pinpoint missing competencies. The system then enters Learning Path & Duration Optimization, employing greedy Hill Climbing and A* Search algorithms to build a structured, high-efficiency study timeline. Finally, under Interview Route Optimization, it tackles the Traveling Salesperson Problem (TSP) using location coordinates and algorithms like Nearest Neighbor or Held-Karp to deliver a final output complete with job-matching results, timelines, and an interactive travel route map.

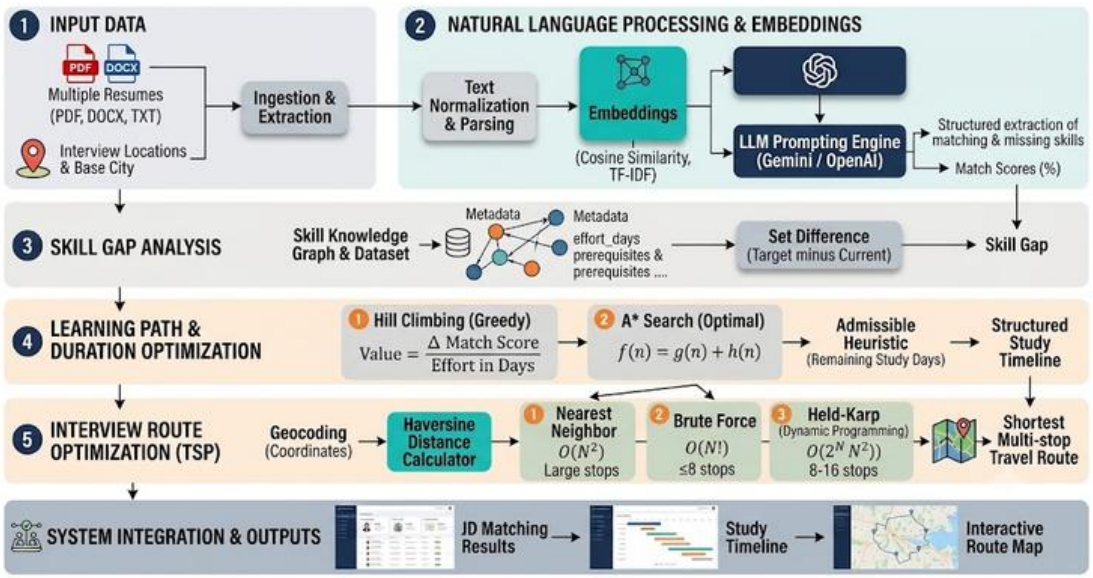


FIG 2.1 PROPOSED SYSTEM ARCHITECTURE

2.1.1 User Interface Layer:

Streamlit provides an interactive user interface with separate tabs or sections for resume upload, job-description matching, and chatbot interaction. It enables users to upload resume documents, enter job descriptions, view analysis results, and interact with the chatbot through a simple and user-friendly interface.

2.1.2 Document Ingestion Layer:

PyPDF extracts text from each uploaded PDF resume. The application associates the extracted content with the original filename or another source identifier to maintain document identity and support accurate retrieval of candidate information during subsequent processing.

2.1.3 Preprocessing and Chunking Layer:

The extracted text undergoes preprocessing to clean and normalize the content. Chunking divides lengthy documents into smaller passages suitable for embedding generation and retrieval. The chunk size and overlap are configured to preserve relevant context while improving retrieval precision and ensuring that important information is not lost between consecutive chunks.

2.1.4 Embedding Generation Layer:

HuggingFace Sentence Transformers, using the all-MiniLM-L6-v2 model, converts each text chunk into a dense numerical vector. These embeddings represent the semantic meaning of the text and enable the application to

identify relevant information based on contextual similarity rather than relying solely on exact keyword matches.

2.1.5 Vector Storage Layer:

ChromaDB stores the generated embeddings, corresponding text chunks, and associated source metadata. Persistent storage allows the application to reuse the existing vector index across sessions, depending on the database configuration and document management logic. This facilitates efficient semantic search and retrieval of relevant resume information.

2.1.6 Job-Description Matching Layer:

The job description and candidate resume content are processed to identify relevant evidence and compare candidate qualifications against job requirements. The retrieved information is used by the Large Language Model (LLM) to generate a match score and an explanation of the candidate's suitability. The exact scoring methodology and implementation are determined by the logic defined in jd_matcher.py.

2.1.7 Retrieval-Augmented Generation (RAG) Chatbot Layer:

When a user submits a question, the application converts it into a retrieval query and searches ChromaDB for relevant text chunks. The retrieved context is supplied to the LLM along with an appropriate prompt to generate a contextually relevant response. Source metadata helps distinguish between different candidates and associate the retrieved

information with the corresponding resume files, improving the traceability of chatbot responses.

2.1.8 Results Presentation Layer:

Streamlit presents the processed results in an organized and interactive format. The application displays candidate rankings, detailed resume analyses, matching and missing skills, improvement suggestions, and conversational answers generated by the chatbot. This enables users to evaluate candidate suitability, identify skill gaps, and obtain useful recommendations for improving resumes in relation to specific job descriptions.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

COMPONENT	MINIMUM SPECIFICATION	RECOMMENDED SPECIFICATION
Processor	Dual-core CPU	Intel Core i5 / AMD Ryzen 5 or equivalent
RAM	4 GB	8 GB or higher
Storage	1 GB free space, plus resume files	5 GB or more for datasets and indexes
GPU	Not required for a small CPU-based prototype	Optional, depending on the embedding or ML model
Network	Internet access for package/model downloads if required	Stable broadband connection

3.2 SOFTWARE REQUIREMENTS

SOFTWARE / TOOL	PURPOSE
Python	Main programming language
pyresparser	Resume parsing and field extraction
LangChain	Orchestration of embedding, retrieval or question-answering components where used

CHAPTER 4
SYSTEM DESIGN AND IMPLEMENTATION

4.1 SYSTEM MODULES

4.1.1 Resume Parsing and Text Extraction

The resume parser accepts PDF, DOCX, and TXT files. The implementation notes identify pypdf and pdfplumber for PDF extraction and python-docx for DOCX extraction. Extracted text is normalized before downstream analysis. This stage converts different document formats into a common text representation. Extraction quality depends on the document structure; scanned PDFs may require OCR if text is not embedded, and OCR is not specified in the supplied implementation description.

4.1.2 Resume and Job Description Matching

This module compares each resume with the supplied job description. For multiple resumes, each document is processed separately and receives a match score, after which candidates are sorted from highest to lowest score. The implementation combines vector-based text similarity with structured LLM extraction.

Cosine similarity compares the direction of resume and job-description vectors:

$$\text{cosine}(u, v) = (u \cdot v) / (||u|| \times ||v||)$$

Here, u and v represent the numerical vectors for the resume and job description. A larger cosine similarity indicates greater alignment in the represented text features; it does not independently prove that a candidate possesses a skill. The implementation description mentions TF-IDF and/or embeddings as the vectorization route.

The JD matcher uses a structured prompt through LangChain and a configurable LLM provider such as Gemini, Groq, or OpenAI. The requested output includes a match score out of 100, matching skills, missing skills, and recommendations for improving the resume. Exact provider availability depends on the configured credentials and selected implementation.

4.1.3 Skill-Gap Analysis

The system compares the skills required by the job with the candidate's existing skills. The basic gap can be represented as Required Skills − Acquired Skills. The skills_dataset.json file stores metadata for skills, including effort_days, prerequisites, and category. The prerequisite links form a skill dependency graph.

If a missing skill depends on other skills, the system expands those prerequisites recursively. A skill is treated as learnable when its prerequisite set is contained in the candidate's acquired skill set. This prevents the learning plan from recommending an advanced skill before its required foundations are completed, provided that the prerequisite data is complete and accurate.

4.1.4 Career Path Optimization with Hill Climbing

Hill Climbing is used as a greedy learning-path strategy. At each iteration, the algorithm identifies missing skills whose prerequisites are satisfied, estimates the match-score improvement associated with each skill, divides that improvement by the estimated effort in days, and selects the highest-value option. The process repeats after updating the acquired-skill set.

Value(skill) = estimated increase in JD match score / effort in days

This approach is straightforward and can produce a useful plan quickly. However, it is a local-search strategy and may select a locally attractive skill that does not lead to the best overall plan. It can also stall when a prerequisite provides little immediate score improvement but unlocks a valuable downstream skill.

4.1.5 Career Path Optimization with A Search*

A* search explores candidate learning states using an evaluation function that combines the cost already incurred with an estimate of the remaining cost:

$f(n) = g(n) + h(n)$

g(n) is the accumulated study effort for the skills selected so far, while h(n) estimates the remaining effort needed to satisfy the job's skill requirements. The search selects promising states according to f(n) and seeks a path with minimum total estimated study effort.

The optimality guarantee depends on the heuristic and implementation. For A* to guarantee an optimal path, the heuristic must not overestimate the true remaining cost (admissibility), and the search must handle state costs and reopening/closed states correctly where required. A sum of effort estimates is admissible only when its construction does not double-count work or otherwise overestimate the actual remaining cost. Therefore, the project's optimality claim should be validated against the exact heuristic and code.

4.1.6 Interview Route Optimization

This module receives a base location and the coordinates of interview venues. It computes pairwise geographic distances and chooses an order for visiting the locations. The implementation description includes three

route strategies so that small and medium-sized problems can use more exhaustive optimization while larger problems can use a faster approximation.

4.1.7 Geographic Distance with the Haversine Formula

The Haversine formula estimates the great-circle distance between two points on a spherical Earth from their latitudes and longitudes. Latitude and longitude differences must be expressed in radians.

$a = \sin^2(\Delta\varphi/2) + \cos(\varphi_1)\cos(\varphi_2)\sin^2(\Delta\lambda/2)$

$c = 2\,\mathrm{atan2}(\sqrt{a}, \sqrt{(1-a)})$; $d = R \times c$

R is the assumed Earth radius (approximately 6,371 km), and d is the calculated distance. This is a geographic straight-line distance, not necessarily the actual road distance or travel time. Road-network routing would require a mapping or routing service.

4.1.8 Route Search Algorithms

Nearest Neighbor: starts at the base or current stop and repeatedly chooses the closest unvisited location. Its typical time complexity is $O(n^2)$ for a straightforward implementation. It is fast but does not guarantee the globally shortest route.

Brute-force permutations: evaluates possible visit orders and selects the one with the lowest total distance. The search grows factorially, commonly described as $O(n!)$ (or $O((n-1)!)$ when the starting location is fixed). It is suitable only for a small number of stops but can find the exact optimum when all relevant tours are enumerated correctly.

Held–Karp dynamic programming: represents a subproblem by a subset of visited stops and the final stop in that subset. It reuses solutions to overlapping subproblems, reducing the factorial search to $O(n^2 \times 2^n)$ time

and $O(n \times 2^n)$ space in the standard formulation. It is still exponential, so it is most practical for moderate-sized inputs.

4.1.9 Streamlit User Interface and Results

The project description indicates that the application is launched with `streamlit run app.py`. The interface brings together resume uploads, job-description input, matching outputs, skill-gap and study-timeline results, and an interactive map or route view. The interface is the presentation layer; the quality of its results depends on the parsing, model configuration, dataset, and optimization modules.

4.2 TECHNOLOGY STACK

Technology / Component	Purpose in the project
Python	Core implementation language for parsing, analysis, optimization, and application logic.
Streamlit	Interactive application interface and results presentation.
pypdf / pdfplumber	Text extraction from PDF resumes.
python-docx	Reading text from DOCX resumes.
Plain-text handling	Processing TXT resumes.
Vectorization: TF-IDF and/or embeddings	Representing resume and job-description text numerically for similarity calculation.
scikit-learn or numerical utilities (as configured)	Potential support for text vectorization and similarity; the exact configured implementation should be confirmed from source code.
LangChain	Prompt orchestration for structured LLM-based matching.
Gemini / Groq / OpenAI	Configurable LLM providers named in the implementation description.
JSON skill dataset	Stores skills, effort estimates, categories, and prerequisites.
Python itertools / dynamic programming	Permutation enumeration and Held–Karp-style route optimization, as described.
Latitude/longitude coordinates	Geographic inputs used by the Haversine calculation.
Interactive map component	Displays the proposed interview route in the user interface.

4.3 ALGORITHMS AND TECHNIQUES USED

Problem	Algorithm / technique	Purpose / limitation
Resume–JD similarity	Vector Space Model + cosine similarity	Measures text-vector alignment; does not independently validate skills.
Structured skill extraction	LLM prompt-based extraction	Produces matched/missing skills and recommendations; can be sensitive to prompts and model errors.
Skill-gap identification	Set difference	Finds required skills absent from the acquired-skill set.
Prerequisite expansion	Recursive graph traversal	Finds foundational skills needed before a target skill.
Greedy learning plan	Hill Climbing	Prioritizes estimated match gain per effort; may reach a local optimum.
Learning-path search	A* search	Uses accumulated cost plus heuristic; optimality depends on heuristic and implementation conditions.
Geographic distance	Haversine formula	Computes spherical great-circle distance from coordinates.
Fast route approximation	Nearest Neighbor	Efficient greedy route; not guaranteed globally optimal.
Exact small-route search	Brute force permutations	Finds minimum among enumerated routes; factorial growth.
Moderate-route optimization	Held–Karp dynamic programming	Uses subset-based memoization; exponential time and memory.

4.4 DATA AND INDEX DESIGN

DATA ITEM	**PURPOSE**
Resume file	Original document supplied for analysis.
Parsed resume record	Extracted fields such as skills, education, experience and contact details, where detected.
Resume text / chunks	Text representation used for embedding and retrieval.
Embedding vectors	Numerical representation of resumes and queries.
FAISS index	Efficient index used to retrieve semantically similar vectors.
Job description	Target role requirements supplied by the user.
Analysis output	Retrieved matches, skill comparison and any configured recommendations.

4.5 RELIABILITY, PRIVACY AND LIMITATIONS

CHALLENGE	HANDLING / RECOMMENDATION
Incorrect parsing	Show extracted fields for review and allow correction where the interface supports it.
Keyword mismatch	Use semantic embeddings alongside explicit skill checks.
Unsupported answers	Ground question answering in retrieved resume text and indicate missing information.
Biased ranking	Do not use similarity alone as a hiring decision; evaluate across varied resumes and roles.
Personal data	Restrict access to resumes, avoid unnecessary retention, and remove files when no longer needed.
Small dataset	Treat results as a prototype; evaluate on a larger, diverse and labelled dataset.
Salary uncertainty	Provide estimates only when a trained model exists, and clearly state uncertainty.

4.6 TESTING AND EVALUATION

Testing should verify that the parser handles supported documents, the embedding pipeline creates vectors, the index retrieves relevant items, and the interface presents understandable results. Evaluation should use a labelled set of job descriptions and resumes where possible.

TEST CASE	EXPECTED RESULT
Upload a valid resume	The system extracts available fields or text.
Resume with missing fields	Missing values are handled without crashing.
Submit a job description	The system produces a query embedding and retrieves relevant results.
Compare related terminology	Semantically related content can be retrieved even without exact wording.
Ask about absent information	The system indicates that the information was not found in the available resume text.
Rebuild or reload index	Stored resumes can be indexed and searched consistently.
Test skill-gap output	Required skills are compared against detected resume skills, with limitations noted.

CHAPTER 5
RESULTS AND DISCUSSIONS

5.1 RESUME UPLOAD:

This user interface screenshot shows the Resume Upload phase of the system, where a user has uploaded multiple PDF resumes for processing. The application is currently updating the vector database and running initial text chunking and embedding generation to prepare the data for downstream matching.

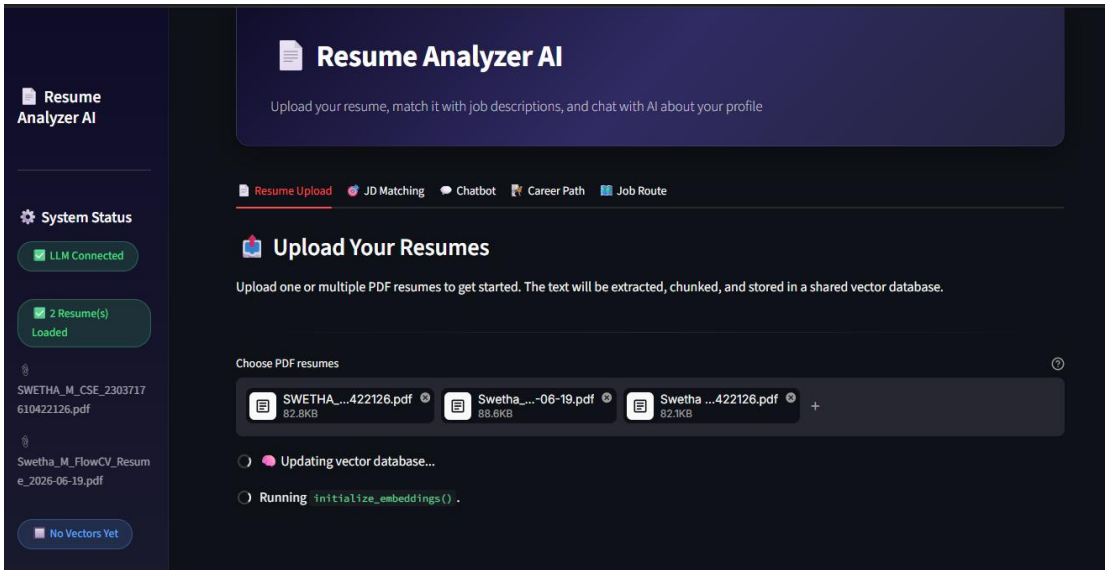


FIG 5.1 UPLOAD RESUME UI

5.2 DATA LOADED

This user interface screenshot shows the Extracted Resume View and Database Statistics panel, allowing users to review the successfully processed, raw text content of a selected profile. Alongside the text preview, the right panel displays real-time backend metrics showing a total of 3 resumes processed into 7,768 characters, 1,023 words, and 24 database chunks.



FIG 5.2

5.3 JD MATCHER

This shows the Job Description Matching phase, where a target job description has been input to analyze and compare against all uploaded candidate profiles. The system is actively executing the LLM Prompting Engine to parse requirements like REST APIs, Git, and software development responsibilities while sequentially matching each candidate resume.

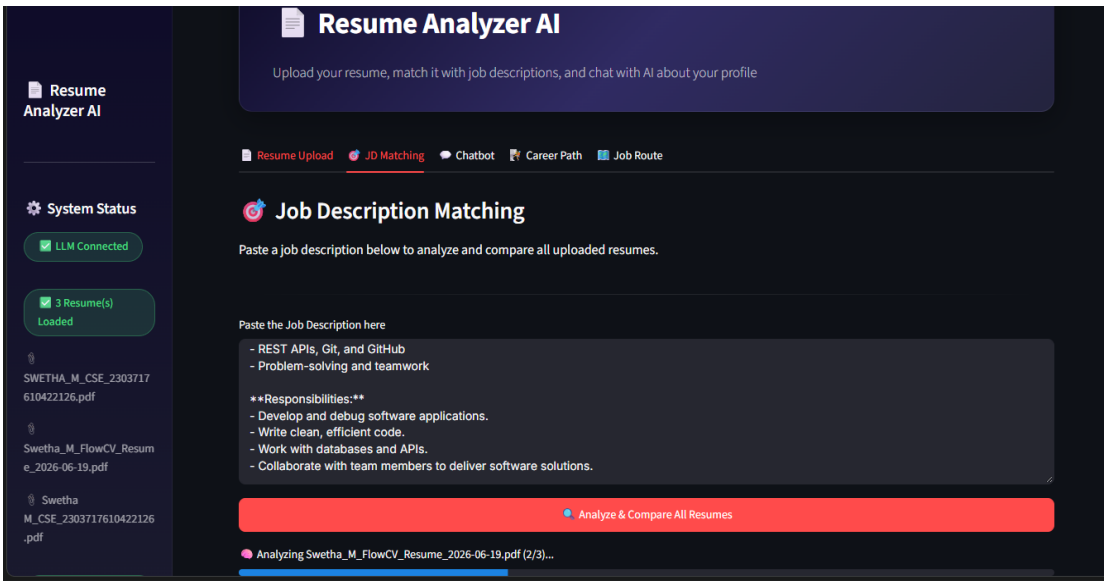


FIG 5.3

5.4 MATCHING SCORE:

Resume Leaderboard and Detailed Feedback panel, displaying the ranked match scores generated for each candidate profile against the job description. The interface highlights an individual candidate's match analysis, breaking down specific overlapping technical qualifications like programming languages and core computer science fundamentals that contributed to a high ranking score.

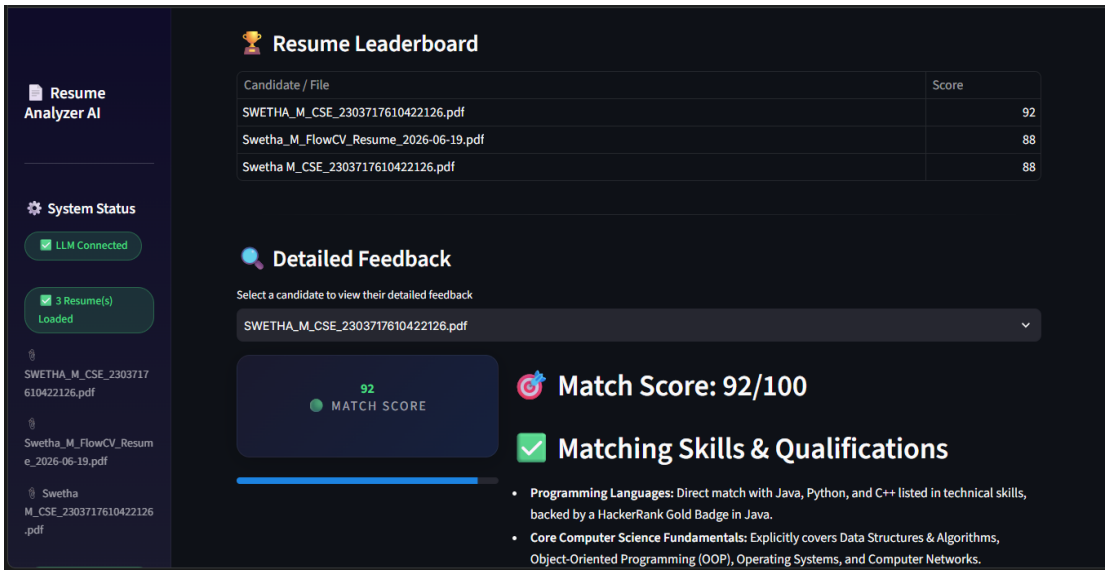


FIG 5.4

5.5 SUGGESTOR

Illustrates how the system isolates specific strengths alongside candidate deficit areas. The dashboard directly evaluates the candidate against requirements, confirming existing strengths in programming, databases, and APIs while flagging missing skills and timeline gaps like software testing frameworks or immediate graduation availability.

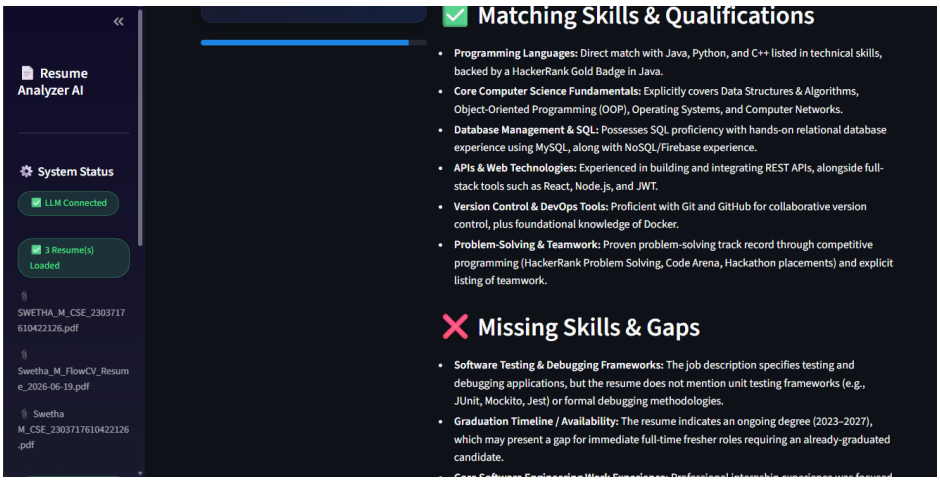


FIG 5.5

5.6 CHATBOT

Interactive Chatbot feature, which allows users to query the system for personalized, conversational insights about their uploaded profiles. The AI is responding to a query about resume issues by generating structured profile critique, specifically flagging an underdeveloped projects section and identifying missing essential blocks like education, internships, and certifications

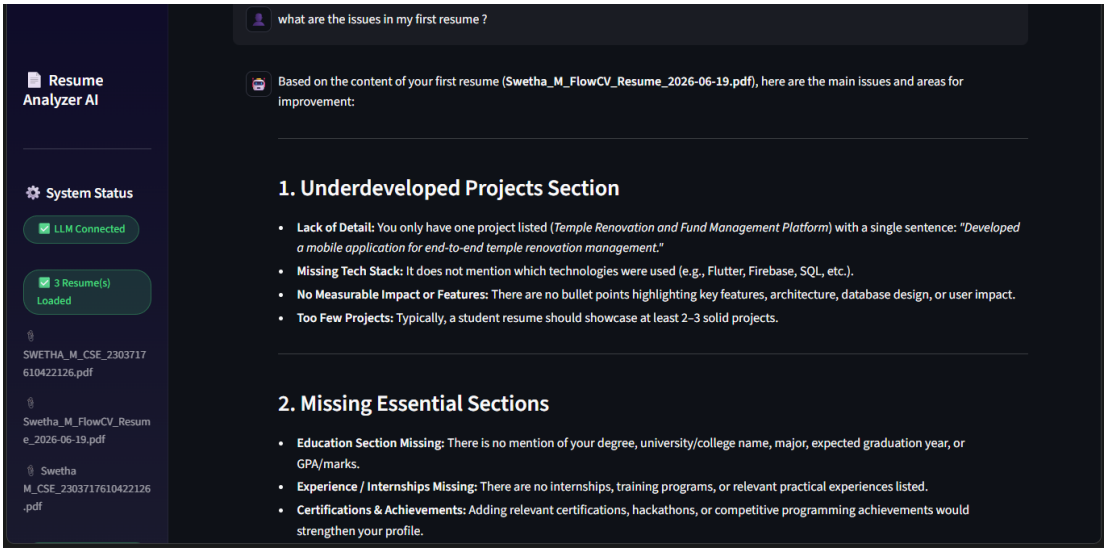


FIG 5.6

5.7 GAPS FINDER AND OPTIMAL STUDY PLAN

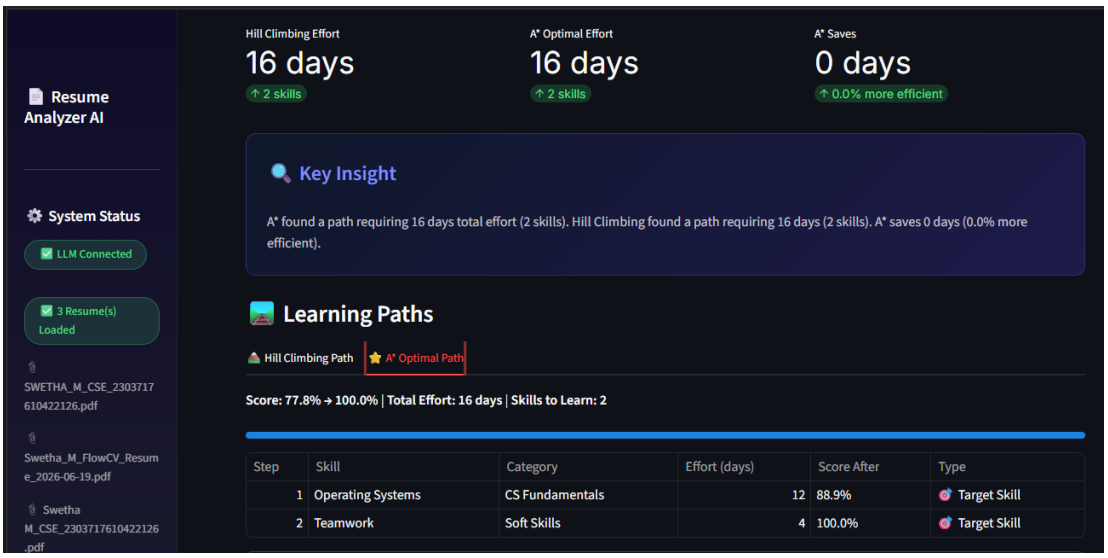


FIG 5.7

5.8 PATH EXPLORED

A* Algorithm State Tracker, which breaks down the step-by-step exploration logic used to calculate the path optimization timeline. The execution log visualizes backend data by tracing nodes explored, queue states, and cost valuations across successive iterations until the optimal goal state is successfully reached.

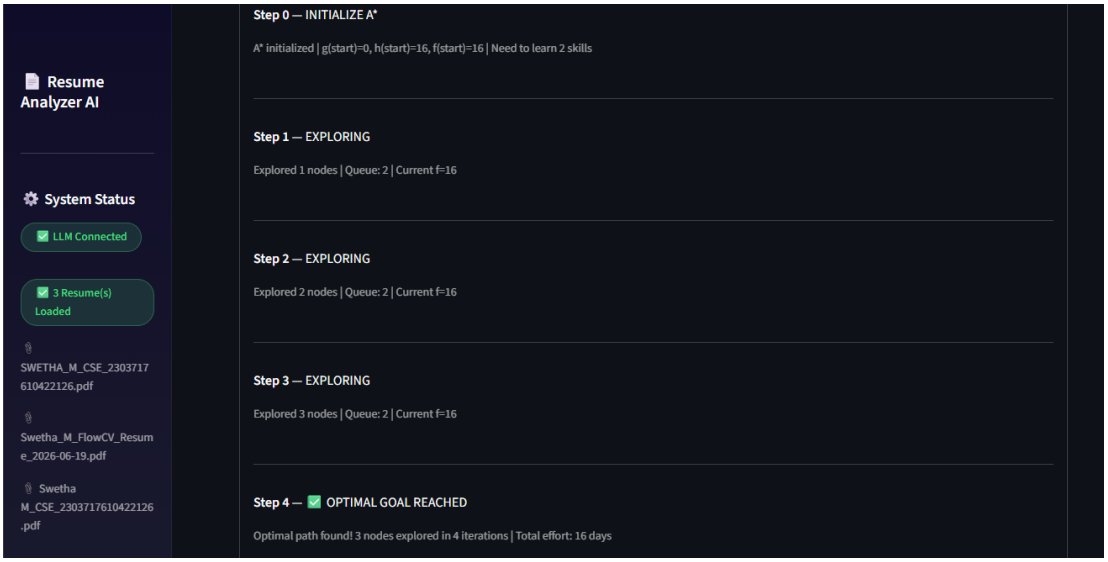


FIG 5.8

5.9 INTERVIEW OPTIMIZED ROUTE FINDER:

This user interface screenshot shows the Interview Route Optimization phase, which handles the spatial routing logic for traveling to different company locations. ThIS displays an Algorithm Comparison for the Traveling Salesperson Problem (TSP), comparing a Nearest Neighbor heuristic against an optimal Brute Force solution.

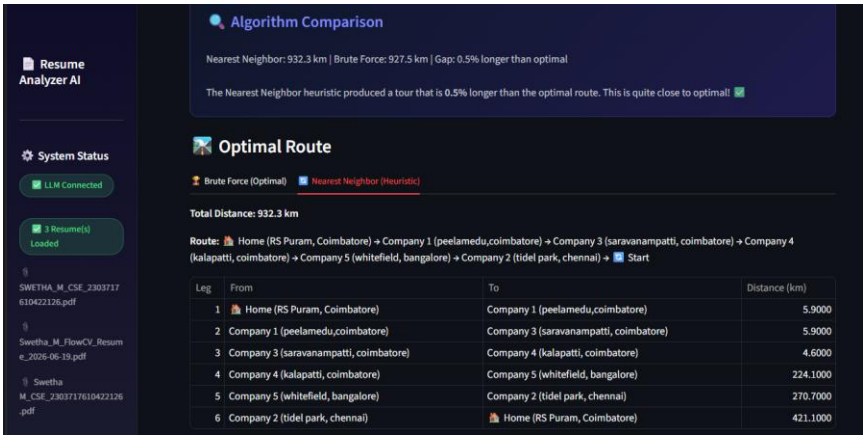


FIG 5.9

5.10 MAP

Interactive Route Map, which visualizes the final geospatial itinerary calculated by the interview routing algorithms. It renders an interactive map view utilizing Leaflet and OpenStreetMap markers to trace the complete physical travel loop between the candidate's home base and various target company locations.

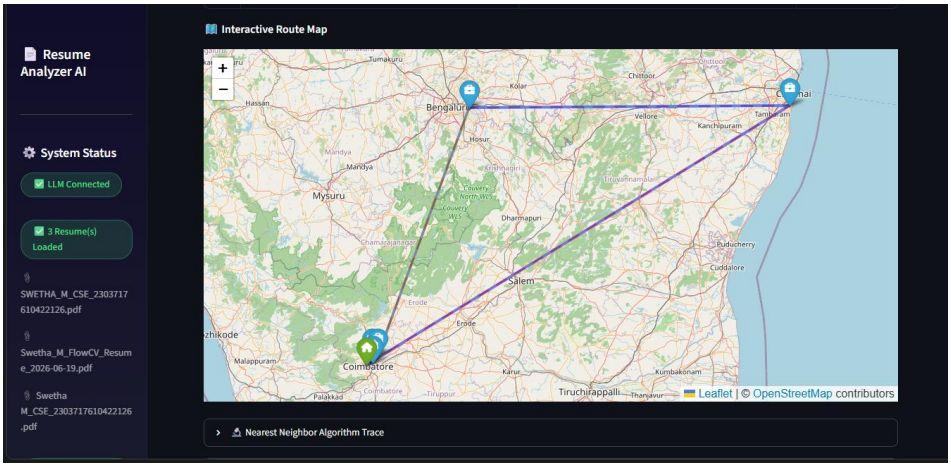


FIG 5.10

CHAPTER 6
CONCLUSION

6.1 CONCLUSION

The Resume Analyzer AI Project combines resume–job-description comparison, skill-gap analysis, prerequisite-aware learning-path planning, and interview-route optimization in one application. Its three primary pipelines use complementary techniques: vector similarity and LLM extraction for resume analysis; skill graphs, Hill Climbing, and A* for study planning; and Haversine distance with Nearest Neighbor, brute force, or Held–Karp for route planning. By presenting the results together, the system can help candidates make more structured decisions about job preparation and interview logistics. The usefulness of the system depends on reliable document extraction, well-maintained skill data, appropriate model configuration, and careful validation of algorithmic assumptions.